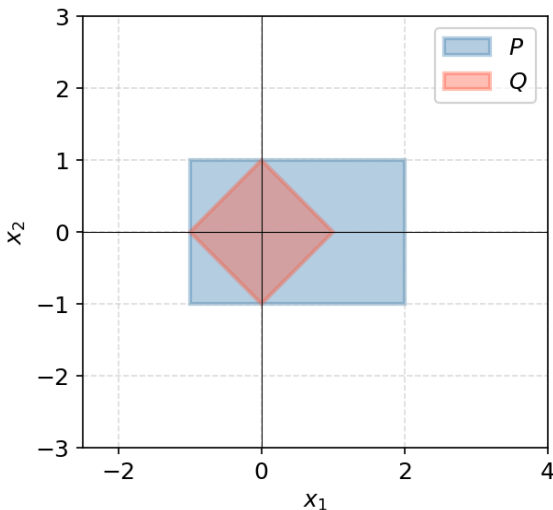


HW5

Problem 1(Code is shown in attached files):

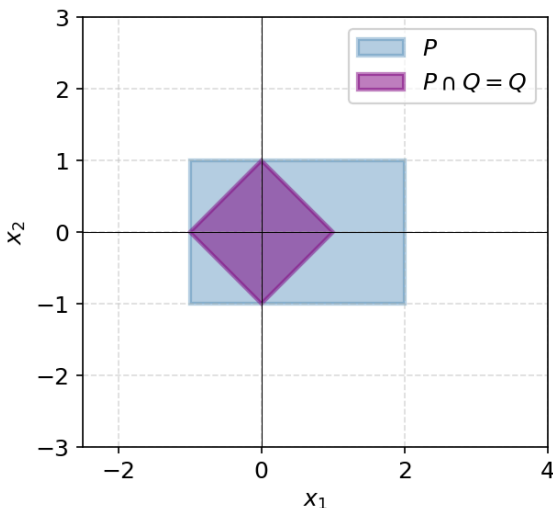
1-a)

The plot of both shapes were done in Python to make it more clean. P is simply a rectangle with x_1 between -1 and 2 and x_2 between -1 and 1. Q is a diamond shape. Since the absolute values of x_1 and x_2 must add up to 1 or less, this means the sides are diagonal at 45 degree angles, centered around the origin. They intersect at the axes, at a value of 1 in all cases.



1-b)

The set intersection of the two sets can be seen simply from the previous plot. It is just the area of set Q , since set Q is entirely enclosed by set P



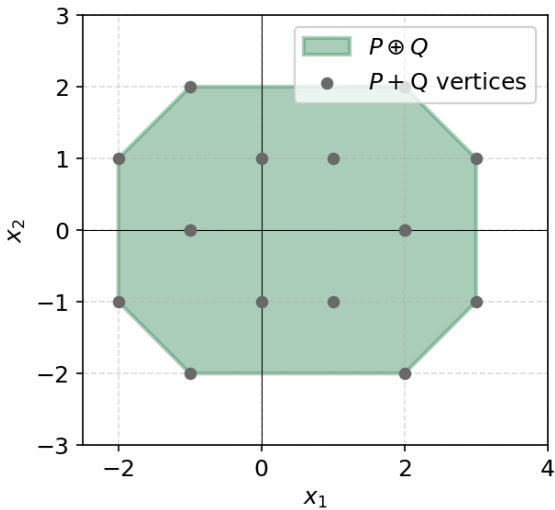
1-c)

The minkoski sum is the set of points spanned by every point in a set, after adding every point in the other set. This can be more simply calculated here by taking the vertices of set Q and

adding them to the vertices of set P, as these will be the most extreme values for the set. This was done using a for loop, as it was not supposed to be done with a built in package:

```
mink_pts = np.array([p + q for p in P_verts for q in Q_verts])
```

In this case, the convex hull minkowski sum leads to the following plot. The perimeter points are plotted, and the outermost ones were used to create the set of the minkoski sum of the P and Q sets. Code used to create the plots are attached to the submission.



Problem 2:

2-a)

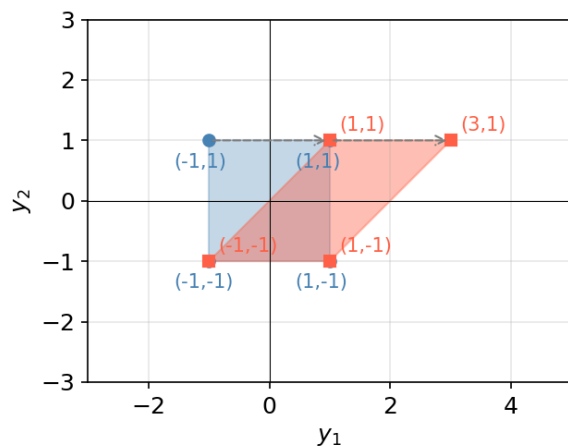
The image can be calculated by taking the vertices of the set and running them through the linear transformation. Since the set is made up of lines and the transformation is linear, the vertices can be directly connected with straight lines after the vertex transformation.

$$x = [-1, -1] \rightarrow y = [-1, -1]$$

$$x = [1, -1] \rightarrow y = [1, -1]$$

$$x = [1, 1] \rightarrow y = [3, 1]$$

$$x = [-1, 1] \rightarrow y = [1, 1]$$



X (domain)
 Y = AX+b (image)

This leads to the following inequalities:

$$-1 \leq y_2 \leq 1$$

$$y_2 \leq y_1 \leq y_2 + 2$$

2-b)

The preimage can be calculated the same way, but rather than applying the forward

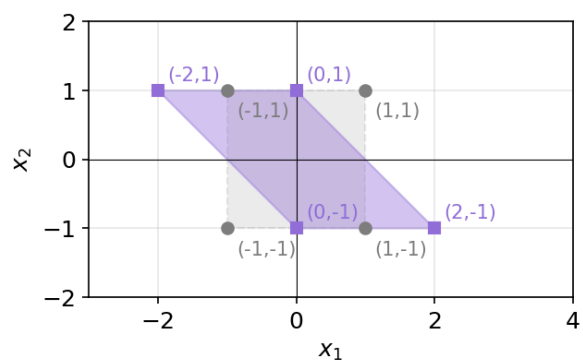
transformation: $y = Ax + b$, the reverse is used: $A^{-1}(y - b) = x$, where y is the domain and x is the preimage

$$y = [0, -1] \rightarrow x = [0, -1]$$

$$y = [2, -1] \rightarrow x = [2, -1]$$

$$y = [2, 1] \rightarrow x = [0, 1]$$

$$y = [0, 1] \rightarrow x = [-2, 1]$$



X (unit square, ref)
 Preimage of S

This leads to the following inequalities:

$$-1 \leq x_1 + x_2 \leq 1$$

$$-1 \leq x_2 \leq 1$$

Problem 3)

3-a)

$$K1 = Pre(Xf) = [-0.250 - 0.5, 0.250 - 0.5] = [-0.750, 0.750]$$

→ *X is between* $[-0.75, 0.75]$

3-b)

$$K2 = Pre(K1) = [-0.750 - 0.5, 0.750 - 0.5] = [-1.250, 1.250]$$

→ *X is between* $[-1.25, 1.25]$

3-c)

$$K3 = Pre(K2) = [-1.250 - 0.5, 1.250 - 0.5] = [-1.750, 1.750]$$

→ *X is between* $[-1.75, 1.75]$

The pattern is that the predecessor state is simply $[-0.5, +0.5]$ added to the previous state.

This is because this is the control input and the system simply adds that to the state when performing discrete time updates.

3-d)

$$R1 = Reach(X0) = [-0.250 - 0.5, 0.250 + 0.5] = [-0.750, 0.750]$$

→ *X is between* $[-0.75, 0.75]$

$$R2 = Reach(R1) = [-0.750 - 0.5, 0.750 + 0.5] = [-1.250, 1.250]$$

→ *X is between* $[-1.25, 1.25]$

3-e)

The controllable sets are the sets which can be controlled to a target. The reachable sets are the sets that can be reached, starting from a target. They are conceptually different, because one goes forward in time while the other goes backwards. In this case, they are the same, because the system dynamics are extremely simple and we can simply add a control input to the states. In more complex systems, this would not be the case and the reachable set would not be the same as the controllable set.

Problem 4:

4-a)

Positive invariance means any starting state in the set will stay within the set indefinitely.

$$a = 0.5 \rightarrow x_{\{k+1\}} = 0.5 * x_k$$

As long as X is in $[-1,1]$,

$$|x_{\{k+1\}}| = |0.5| * |x_k| \leq 0.5$$

This continues indefinitely, with values within the set getting asymptotically close to 0, which is within the set. Thus, it is positively invariant.

4-b)

$$a = 1 \rightarrow x_{\{k+1\}} = 1 * x_k$$

As long as X is in $[-1,1]$,

$$|x_{\{k+1\}}| = |1| * |x_k| \leq 1$$

This continues indefinitely, with values within the set staying exactly where they are, which is within the set. Thus, it is positively invariant.

4-c)

$$a = 1 \rightarrow x_{\{k+1\}} = 1.2 * x_k$$

As long as X is in $[-1,1]$,

$$|x_{\{k+1\}}| = |1.2| * |x_k| \leq 1.2$$

Values within the set immediately leave the set, so it is not positively invariant. The only way for this to be positively invariant is if applying the forward transformation does not increase the state. This occurs only for $x = 0$, so this would only be positively invariant for $\alpha = [0, 0]$

4-d)

For a finite constraint set in an autonomous system, positive invariance can only be achieved if state variables do not grow indefinitely, meaning that a must be 1 or less. In general cases, this means eigenvalues must be one or less, as these would be pure scaling magnitudes along the eigenvalues, which by definition would have to be one or less to avoid increasing the state variables.

Problem 5:

5-a)

If $x_k = 22$, then it is within the set, but this leads to:

$$x_{x+1} = 1.2x_k + u_k = 1.2 * 22 + [-0.3, 0.3] = 2.4 + [-0.3, 0.3] = [2.1, 2.7]$$

This is outside the set, so it is not control invariant

5-b,c)

For control invariance to be just barely achieved, the coefficient multiplied by state must equal the state plus minkowski sum with the achievable control.

$$x_{x+1} = x_{crit} = 1.2x_k + u_k = 1.2 * x_{crit} + [-0.3, 0.3] \rightarrow 0.2x_{crit} = [-0.3, 0.3]$$

$$x_{crit} = [-0.3, 0.3]/0.2 = [-1.5, 1.5]$$

Thus, the set is from [-1.5,1.5]

5-d)

$$x_{x+1} = 1.2x_k + u_k = 1.2 * 1.5 + [-0.3, 0.3] = 1.8 + [-0.3, 0.3] = [1.5, 2.1]$$

Using a control input of -0.3 yields an $x_{x+1} = 1.5$, meaning this is positively invariant.

Problem 6)

Rather than using mpt3 in Matlab, the polytope library in python was used, as I don't have as easy access to Matlab, and I want to learn how to use libraries that I am likely to use in the future.

6-a,b)

After importing packages, the system was defined. Calculating predecessor sets was done with the following functions:

Code:

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import polytope as pc
import os

plt.rcParams.update({'font.size': 12})
os.makedirs("HW5P6", exist_ok=True)

A = np.array([[1.0, 1.0], [0.0, 1.0]])
B = np.array([[0.5], [1.0]])

X = pc.box2poly([[-5, 5], [-2, 2]])
U = pc.box2poly([[-1, 1]])
Xf = pc.box2poly([[-0.5, 0.5], [-0.5, 0.5]])

def pre(T):
    n, m = A.shape[0], B.shape[1]
    HAB = np.hstack([T.A @ A, T.A @ B])
    HX = np.hstack([X.A, np.zeros((X.A.shape[0], m))])
    HU = np.hstack([np.zeros((U.A.shape[0], n)), U.A])
    H = np.vstack([HAB, HX, HU])
    h = np.concatenate([T.b, X.b, U.b])
    return pc.projection(pc.Polytope(H, h), list(range(1, n + 1)))

K1 = pre(Xf)
K2 = pre(K1)
K3 = pre(K2)
```

It was then plotted with this code:


```

def poly_area(p):
    v = pc.extreme(p)
    order = np.argsort(np.arctan2(v[:,1] - v[:,1].mean(), v[:,0] -
v[:,0].mean()))
    v = v[order]
    # shoelace
    x, y = v[:,0], v[:,1]
    return 0.5 * abs(np.dot(x, np.roll(y,-1)) - np.dot(y, np.roll(x,-1)))

areas = {name: poly_area(p) for name, p in [('$X_f$', Xf), ('$K_1$', K1),
('$K_2$', K2), ('$K_3$', K3)]}

def plot_poly(ax, poly, color, alpha):
    verts = pc.extreme(poly)
    order = np.argsort(np.arctan2(verts[:,1] - verts[:,1].mean(),
verts[:,0] - verts[:,0].mean()))
    verts = verts[order]
    ax.fill(verts[:,0], verts[:,1], color=color, alpha=alpha)
    ax.plot(np.append(verts[:,0], verts[0,0]),
np.append(verts[:,1], verts[0,1]), color=color, lw=1)

patches = [mpatches.Patch(color='limegreen', label='$X_f$'),
mpatches.Patch(color='steelblue', label='$K_1$'),
mpatches.Patch(color='royalblue', label='$K_2$'),
mpatches.Patch(color='navy', label='$K_3$')]

fig, axes = plt.subplots(1, 2, figsize=(9.0, 5.0))

# Left: nested controllable sets
ax = axes[0]
for poly, color, alpha in
[(K3, 'navy', 0.45), (K2, 'royalblue', 0.45), (K1, 'steelblue', 0.55), (Xf, 'limegre
en', 0.9)]:
    plot_poly(ax, poly, color, alpha)
ax.set_xlabel("$x_1$"); ax.set_ylabel("$x_2$")
ax.set_xlim(-5, 5); ax.set_ylim(-2, 2)
ax.grid(True, alpha=0.3)
ax.legend(handles=patches, loc='lower center', bbox_to_anchor=(0.5, 0.02),
bbox_transform=fig.transFigure, ncol=2)

```

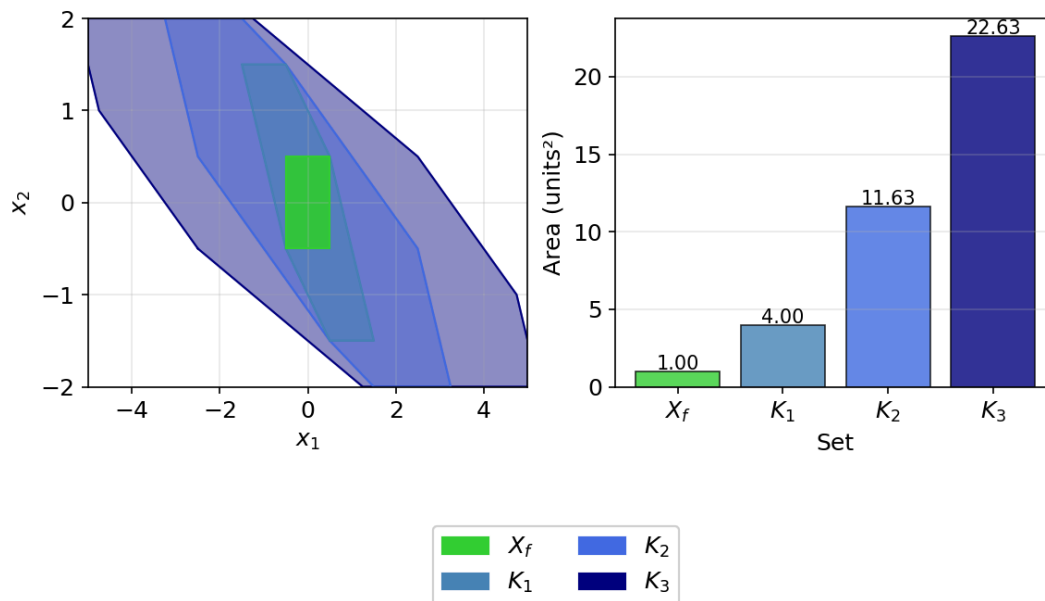
```

# Right: exact area growth
ax2 = axes[1]
labels, vals = list(areas.keys()), list(areas.values())
bar_colors = ['limegreen', 'steelblue', 'royalblue', 'navy']
ax2.bar(labels, vals, color=bar_colors, alpha=0.8, edgecolor='k', lw=0.8)
for i, v in enumerate(vals):
    ax2.text(i, v + 0.1, f"{v:.2f}", ha='center', fontsize=10)
ax2.set_ylabel("Area (units²)")
ax2.set_xlabel("Set")
ax2.grid(True, alpha=0.3, axis='y')
ax2.legend(handles=patches, loc='lower center', bbox_to_anchor=(0.5,
0.02), bbox_transform=fig.transFigure, ncol=2)

plt.subplots_adjust(bottom=0.35)
plt.savefig("HW5P6/controllable_sets_K1_K2_K3.png", dpi=150)
plt.close()

```

This lead to the following plot:



6-c)

The N-step controllable set generally grows as N increases, because there is a spectrum of allowable control inputs, which will lead to a spectrum of outputs, even from a single starting state. This set of outputs will increase further if the starting state is a set of possible states, as

each of those will then have a variety of outcomes, increasing even further as the set becomes more and more broad. This increase in controllable sets is almost guaranteed to increase, as increasing the number of timesteps available should almost always give a larger set of possible outcomes from the control.

6-d)

The set will eventually stop growing, if there are bounds to the set, such as state constraints or if the system is not controllable everywhere. If $K_{n+1} = K_n$, this means that the controllable set is not increasing, meaning every possible outcome from the control is already present in the set and adding additional timesteps will not change where the system may end up. This would correspond to the controllable set containing every possible outcome that is possible, given the dynamics and system.